

Task 1 – Design Parking System

AIM

To design and implement a Java program for a parking system that manages different types of parking spaces and checks whether a vehicle can be parked.

ALGORITHM

1. Start the program.
2. Create a parking system with spaces for **big, medium, and small** vehicles.
3. Initialize the number of available spaces for each vehicle type.
4. When a vehicle arrives, identify its vehicle type.
5. Check whether a parking space of that type is available.
6. If a space is available, park the vehicle and decrease the available space count.
7. If no space is available, return false.
8. Display whether the vehicle was successfully parked.
9. Stop the program.

PROGRAM

```
import java.util.Scanner;
```

```
class ParkingSystem {
```

```
    private int big;
```

```
    private int medium;
```

```
    private int small;
```

```
    public ParkingSystem(int big, int medium, int small) {
```

```
        this.big = big;
```

```
        this.medium = medium;
```

```
        this.small = small;
```

```
    }
```

```
public boolean addCar(int carType) {
```

```
    if (carType == 1 && big > 0) {
```

```
        big--;
```

```
        return true;
```

```
    }
```

```
    if (carType == 2 && medium > 0) {
```

```
        medium--;
```

```
        return true;
```

```
    }
```

```
    if (carType == 3 && small > 0) {
```

```
        small--;
```

```
        return true;
```

```
    }
```

```
    return false;
```

```
}
```

```
public static void main(String[] args) {
```

```
    Scanner sc = new Scanner(System.in);
```

```
    System.out.print("Enter number of big spaces: ");
```

```
    int big = sc.nextInt();
```

```
    System.out.print("Enter number of medium spaces: ");
```

```
int medium = sc.nextInt();

System.out.print("Enter number of small spaces: ");
int small = sc.nextInt();

ParkingSystem parking = new ParkingSystem(big, medium, small);

System.out.print("Enter car type (1-Big, 2-Medium, 3-Small): ");
int carType = sc.nextInt();

boolean result = parking.addCar(carType);

System.out.println("Output: " + result);

sc.close();
}
}
```

OUTPUT

Enter number of big spaces: 1
Enter number of medium spaces: 1
Enter number of small spaces: 0
Enter car type (1-Big, 2-Medium, 3-Small): 1
Output: true

RESULT

Thus, the Java program to design and implement a parking system for different types of vehicles was successfully executed and the correct output was obtained.

Task 2 – Design Underground System

AIM

To design and implement a Java program for an underground transportation system that records passengers' check-in and check-out details and calculates the average travel time between two stations.

ALGORITHM

1. Start the program.
2. Create a system to store passenger check-in information.
3. When a passenger checks in, store their ID, station name, and check-in time.
4. When the passenger checks out, retrieve their check-in details.
5. Calculate the travel time between the check-in and check-out stations.
6. Store the total travel time and number of trips for each pair of stations.
7. Calculate the average travel time using:
Average Time = Total Travel Time / Number of Trips
8. Display the average travel time.
9. Stop the program.

PROGRAM

```
import java.util.*;
```

```
class UndergroundSystem {
```

```
    static class CheckIn {
```

```
        String station;
```

```
        int time;
```

```
        CheckIn(String station, int time) {
```

```
            this.station = station;
```

```
            this.time = time;
```

```
        }
```

```
}
```

```
static class TravelData {
```

```
    int totalTime;
```

```
    int totalTrips;
```

```
    TravelData(int totalTime, int totalTrips) {
```

```
        this.totalTime = totalTime;
```

```
        this.totalTrips = totalTrips;
```

```
    }
```

```
}
```

```
private Map<Integer, CheckIn> checkIns = new HashMap<>();
```

```
private Map<String, TravelData> trips = new HashMap<>();
```

```
public void checkIn(int id, String stationName, int t) {
```

```
    checkIns.put(id, new CheckIn(stationName, t));
```

```
}
```

```
public void checkOut(int id, String stationName, int t) {
```

```
    CheckIn checkIn = checkIns.get(id);
```

```
    String route = checkIn.station + "->" + stationName;
```

```
    int travelTime = t - checkIn.time;
```

```
    if (trips.containsKey(route)) {
```

```
        TravelData data = trips.get(route);
```

```
        data.totalTime += travelTime;
```

```

        data.totalTrips++;
    } else {
        trips.put(route, new TravelData(travelTime, 1));
    }

    checkIns.remove(id);
}

public double getAverageTime(String startStation, String endStation) {
    String route = startStation + "->" + endStation;
    TravelData data = trips.get(route);

    return (double) data.totalTime / data.totalTrips;
}

public static void main(String[] args) {
    UndergroundSystem system = new UndergroundSystem();

    system.checkIn(1, "A", 3);
    system.checkOut(1, "B", 8);

    system.checkIn(2, "A", 10);
    system.checkOut(2, "B", 20);

    double result = system.getAverageTime("A", "B");

    System.out.println("Average travel time: " + result);
}

```

```
}
```

OUTPUT

Average travel time: 7.5

RESULT

Thus, the Java program to design an underground transportation system and calculate the average travel time between stations was successfully executed and the correct output was obtained.

Task 3 – Design Browser History

AIM

To design and implement a Java program to simulate browser history operations such as visiting a new URL, moving backward, and moving forward through the browsing history.

ALGORITHM

1. Start the program.
2. Initialize the browser with a homepage.
3. Create a data structure to store visited web pages.
4. When a new URL is visited, add it to the history.
5. When the **back** operation is performed, move to the previous page up to the specified number of steps.
6. When the **forward** operation is performed, move to the next page up to the specified number of steps.
7. Display the current page after each operation.
8. Stop the program.

PROGRAM

```
import java.util.*;
```

```
class BrowserHistory {
```

```
    private List<String> history;
```

```
    private int current;
```

```
    public BrowserHistory(String homepage) {
```

```
        history = new ArrayList<>();
```

```
        history.add(homepage);
```

```
        current = 0;
```

```
    }
```

```
public void visit(String url) {  
    // Remove forward history  
    while (history.size() > current + 1) {  
        history.remove(history.size() - 1);  
    }
```

```
    history.add(url);  
    current++;  
}
```

```
public String back(int steps) {  
    current = Math.max(0, current - steps);  
    return history.get(current);  
}
```

```
public String forward(int steps) {  
    current = Math.min(history.size() - 1, current + steps);  
    return history.get(current);  
}
```

```
public static void main(String[] args) {
```

```
    BrowserHistory browser = new BrowserHistory("leetcode.com");
```

```
    browser.visit("google.com");
```

```
    browser.visit("facebook.com");
```

```
    browser.visit("youtube.com");
```

```
System.out.println("Back 1: " + browser.back(1));  
System.out.println("Back 1: " + browser.back(1));  
System.out.println("Forward 1: " + browser.forward(1));  
  
browser.visit("linkedin.com");  
  
System.out.println("Forward 2: " + browser.forward(2));  
System.out.println("Back 2: " + browser.back(2));  
}  
}
```

OUTPUT

Back 1: facebook.com
Back 1: google.com
Forward 1: facebook.com
Forward 2: linkedin.com
Back 2: google.com

RESULT

Thus, the Java program to design and implement a browser history system with visit, back, and forward operations was successfully executed and the correct output was obtained.

Task 4 – Design HashSet

AIM

To design and implement a Java program that performs HashSet operations such as add, remove, and contains without storing duplicate elements.

ALGORITHM

1. Start the program.
2. Create a HashSet to store integer elements.
3. Add elements to the HashSet.
4. Since HashSet stores only unique elements, duplicate values are ignored.
5. Check whether a given element exists using contains().
6. Remove an element using remove().
7. Display the HashSet before and after removal.
8. Stop the program.

PROGRAM

Java

```
import java.util.HashSet;
```

```
import java.util.Scanner;
```

```
class DesignHashSet {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        HashSet<Integer> set = new HashSet<>();
```

```
        System.out.print("Enter number of elements to add: ");
```

```
        int n = sc.nextInt();
```

```
        System.out.println("Enter elements:");
```

```
for (int i = 0; i < n; i++) {  
    set.add(sc.nextInt());  
}  
  
System.out.println("HashSet Elements: " + set);  
  
System.out.print("Enter element to check: ");  
int check = sc.nextInt();  
  
System.out.println("Contains " + check + ": " + set.contains(check));  
  
System.out.print("Enter element to remove: ");  
int remove = sc.nextInt();  
  
set.remove(remove);  
  
System.out.println("HashSet after removal: " + set);  
  
sc.close();  
}  
}
```

OUTPUT

Enter number of elements to add: 5

Enter elements:

10 20 30 20 40

HashSet Elements: [20, 40, 10, 30]

Enter element to check: 30

Contains 30: true

Enter element to remove: 20

HashSet after removal: [40, 10, 30]

RESULT

Thus, the Java program to design and implement a HashSet using add, remove, and contains operations was successfully executed, and the correct output was obtained.

Task 5 – Throne Inheritance

AIM

To design and implement a Java program to maintain the order of inheritance in a kingdom using birth and death operations and generate the current inheritance order.

ALGORITHM

1. Start the program.
2. Create the king as the first member of the family.
3. Maintain a family tree using a list of children for each person.
4. When a person is born, add the child to the parent's list.
5. When a person dies, mark that person as deceased.
6. Perform a depth-first traversal of the family tree starting from the king.
7. Add living members to the inheritance list.
8. Display the current inheritance order.
9. Stop the program.

PROGRAM

```
import java.util.*;

class ThroneInheritance {

    private String king;
    private Map<String, List<String>> children;
    private Set<String> dead;

    public ThroneInheritance(String kingName) {
        king = kingName;
        children = new HashMap<>();
        dead = new HashSet<>();
        children.put(kingName, new ArrayList<>());
    }
}
```

```
}
```

```
public void birth(String parentName, String childName) {  
    children.putIfAbsent(childName, new ArrayList<>());  
    children.get(parentName).add(childName);  
}
```

```
public void death(String name) {  
    dead.add(name);  
}
```

```
public List<String> getInheritanceOrder() {  
    List<String> result = new ArrayList<>();  
    dfs(king, result);  
    return result;  
}
```

```
private void dfs(String name, List<String> result) {
```

```
    if (!dead.contains(name)) {  
        result.add(name);  
    }
```

```
    for (String child : children.getOrDefault(name, new ArrayList<>())) {  
        dfs(child, result);  
    }  
}
```

```
public static void main(String[] args) {

    ThroneInheritance family =
        new ThroneInheritance("king");

    family.birth("king", "andy");
    family.birth("king", "bob");
    family.birth("king", "catherine");

    family.birth("andy", "matthew");
    family.birth("bob", "alex");
    family.birth("bob", "asha");

    System.out.println("Inheritance Order:");
    System.out.println(family.getInheritanceOrder());

    family.death("bob");

    System.out.println("After Bob's Death:");
    System.out.println(family.getInheritanceOrder());
}
}
```

OUTPUT

Inheritance Order:

[king, andy, matthew, bob, alex, asha, catherine]

After Bob's Death:

[king, andy, matthew, alex, asha, catherine]

RESULT

Thus, the Java program to design and implement a **Throne Inheritance system** using birth, death, and inheritance-order operations was successfully executed, and the correct output was obtained.